Backpropagation is the process by which, neural networks minimize the error in their predicted output by adjusting the weights and biases of their neurons. This is backpropagation in simple terms but how does all these changes take place? how does the error in the hidden layers get calculated? What does calculus have to do with all this? You will get all your questions answered by the end of this article. So lets get started.

Before getting into the details of backpropagation, let us skim through the entire learning process once:

How does learning occur in a neural network?

The process of learning in a neural network takes place in three steps.

- **Step 1:** The input data points are fed into the neural network. This input data flows through the different layers of the neural network and produces an output or prediction in the final output layer. The whole process by which the data flows from the input layer to the output layer is called **forward propagation**. We will see the details of forward propagation below.
- **Step 2:** Now as we have our output, we have to calculate the loss in the output. We have a lot of options for calculating the loss such as the mean squared error, binary cross entropy etc.
- **Step 3:** After calculating the loss, we have to tell the neural network to change it's parameters (weight and bias) in order to minimize the loss. This process is called **backpropagation**.

Forward Propagation in an Artificial Neural Network:

An ANN is fundamentally made up of three types of layers. The input layer, the hidden layers and the output layer. The flow of data through the ANN happens like this:

- In the first pass, that is, the first time when data flows in the forward direction through the network, the input or the features on which the neural network needs to be trained is fed into the neurons of the input layer.
- These input values then get passed through the neurons of the hidden layer where these values are first multiplied by their respective weights and then added with a bias. We can call this the pre activation function.
- A pre activation function is always followed by an activation function. There are a lot of activation functions lying around such as sigmoid, hardtan, relu etc.
- The final layer is the output layer where the calculated output of the neural network shows up.

The Loss function:

After the inputs are forward propagated and an output is produced, we can find out the **error** in the output. The error is the difference between the predicted output and the ground truth.

But in a neural network, we often do not calculate the error in the output. Instead we calculate the loss using specific loss functions, which we later on use in optimization algorithms to bring down the loss to a minimum value.

There are a lot of loss functions to compute the loss such as the mean squared error, binary cross entropy etc. Each of these functions have specific qualities which can be exploited according to the problem at hand.

The Gradient Descent Algorithm:

The whole idea of backpropagation is to minimize the loss. We have a lot of optimization algorithms to do that. But for the sake of simplicity, let us

begin with a basic yet powerful optimization algorithm, the Gradient descent algorithm.

Here the idea is to compute the rate of change of the loss with respect to each parameter, and modify each parameter in the direction of decreasing loss. This is saying that, an unit change in the any of the parameters, let's say the weight, leads to some change in the loss. If the change is negative , then we need to increase weight to decrease the loss, whereas if the change is positive, we need to decrease the weight. We can write this mathematically as,

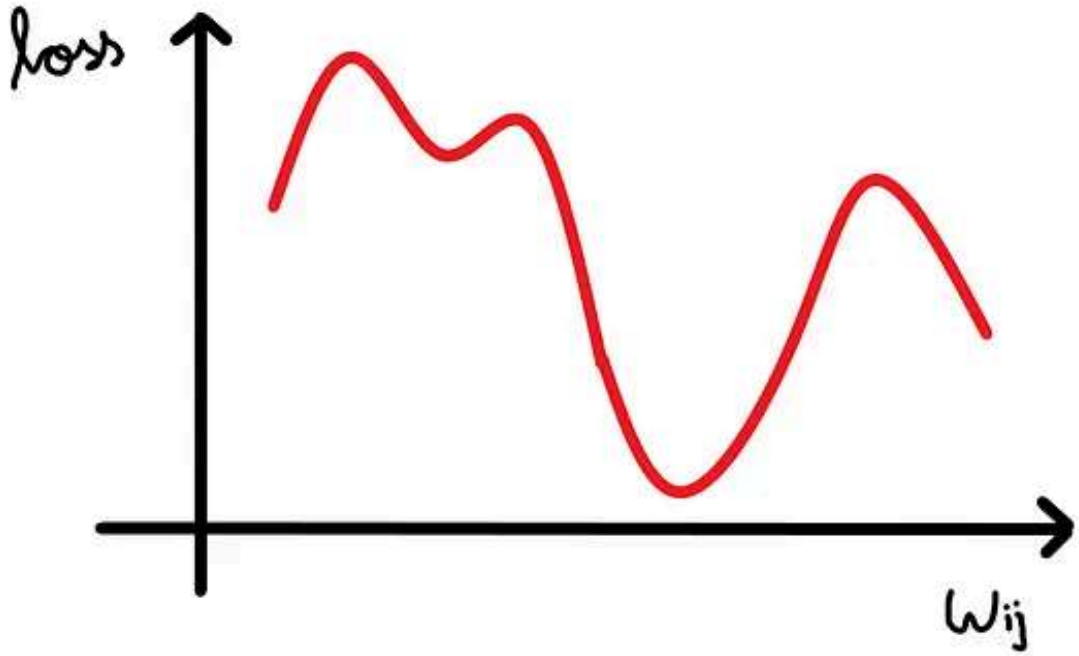
```
new_weight = old_weight - learning_rate * gradient
```

where the gradient is the partial derivative of the loss function with respect to the weight. The learning rate is just a scaling factor that is used to scale up or scale down the gradient. It is explained in more detail in the upcoming texts below. The same formula applies for the bias:

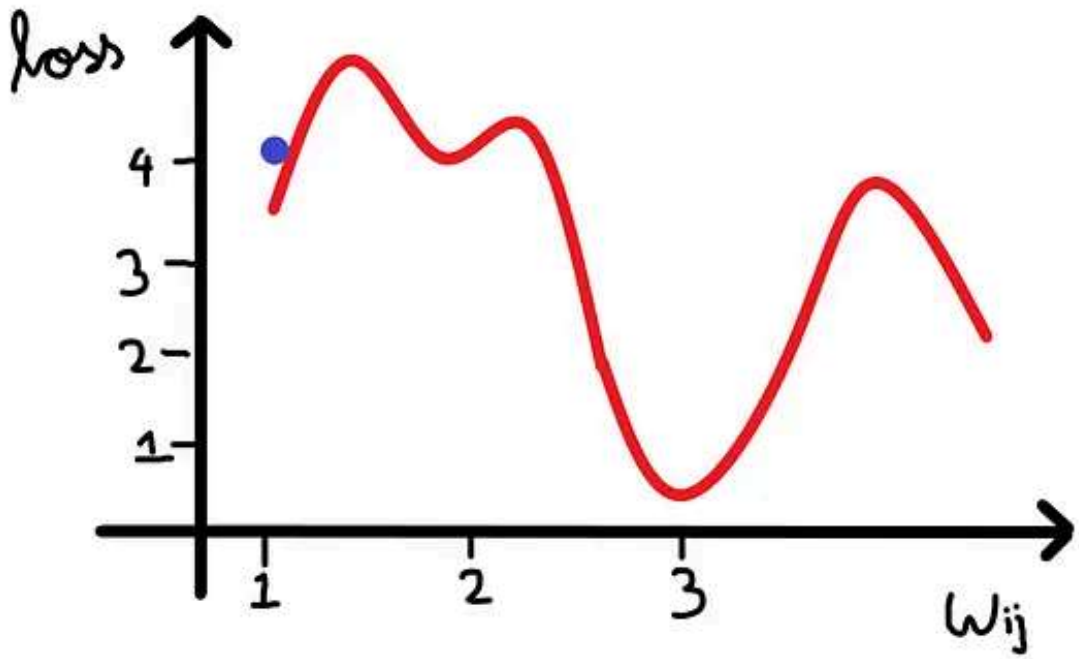
```
new_bias = old_bias - learning_rate * gradient
```

where the gradient is the partial derivative of the loss function with respect to the bias.

Look at the graph below. We have plotted the loss of a neural network with respect to the change in weight of a single neuron.



Now we can see that there are a lot of local minima (all the small downward curves) in the curve, but we are interested in bringing the loss down to the global minima (the biggest downward curve). Suppose our weight value is now somewhere near the origin in our example graph (say 1 and so our loss is somewhere near 4).

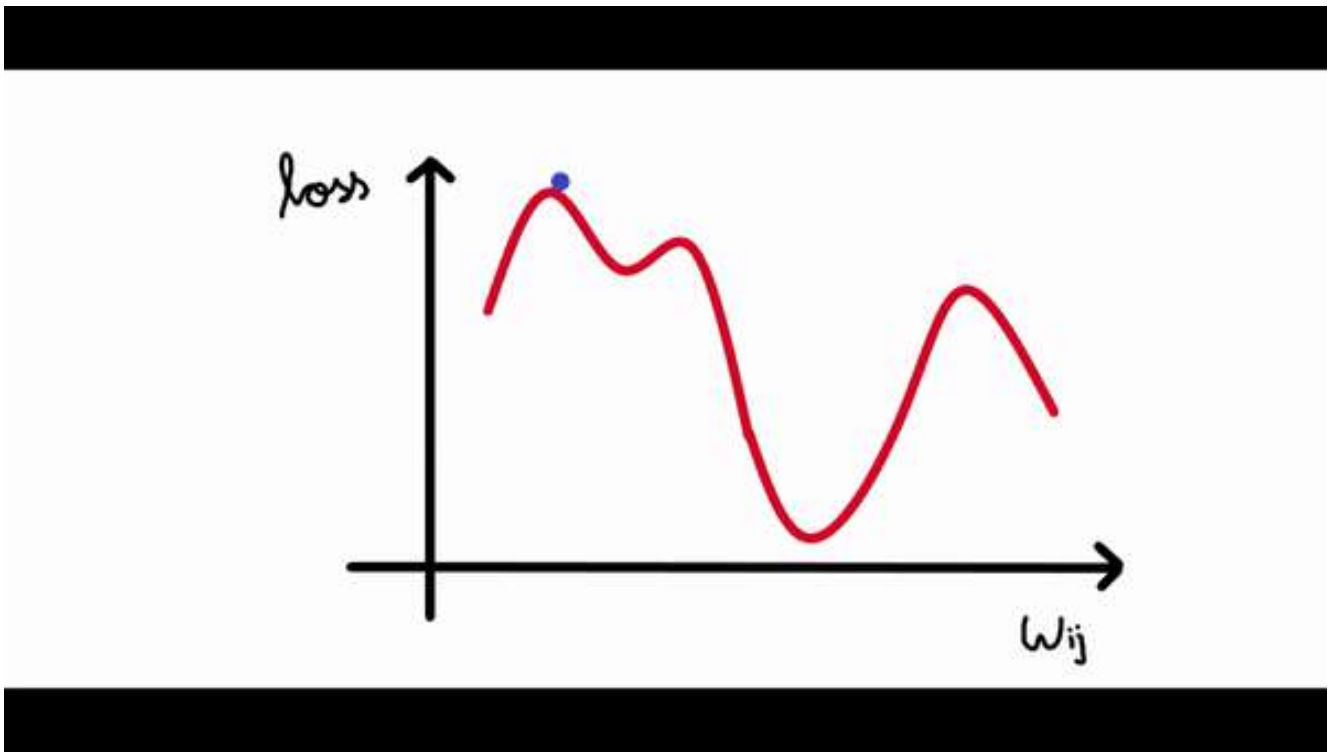


We have to bring the weight value to approximately 3 so that the loss is minimum.

We also have to bear in mind that, the weight should be changed by a value which is proportional to the loss. That is why the gradient is given by the partial differentiation of the loss with respect to the weight. And so the steps in the gradient descent algorithm are:

- The gradient (partial differentiation of the loss function with respect to the weight(or bias)) is calculated
- The gradient is multiplied by a learning rate.
- The gradient multiplied by the learning rate is then subtracted from the weight (or bias).

This sequence of actions is repeated several times till the loss gets converged to the global minimum.



More about the Gradient:

The gradient of a straight line is usually calculated by using the general slope formula:

$$m = \frac{rise}{run} = \frac{y_2 - y_1}{x_2 - x_1}$$

Two points which are separated from each other by a certain distance on the line is taken and the slope is calculated. This method of calculating the gradient gives precise calculations when the graph is a straight line. But when we have uneven curves, rise by run might not be a good idea to calculate the gradient.

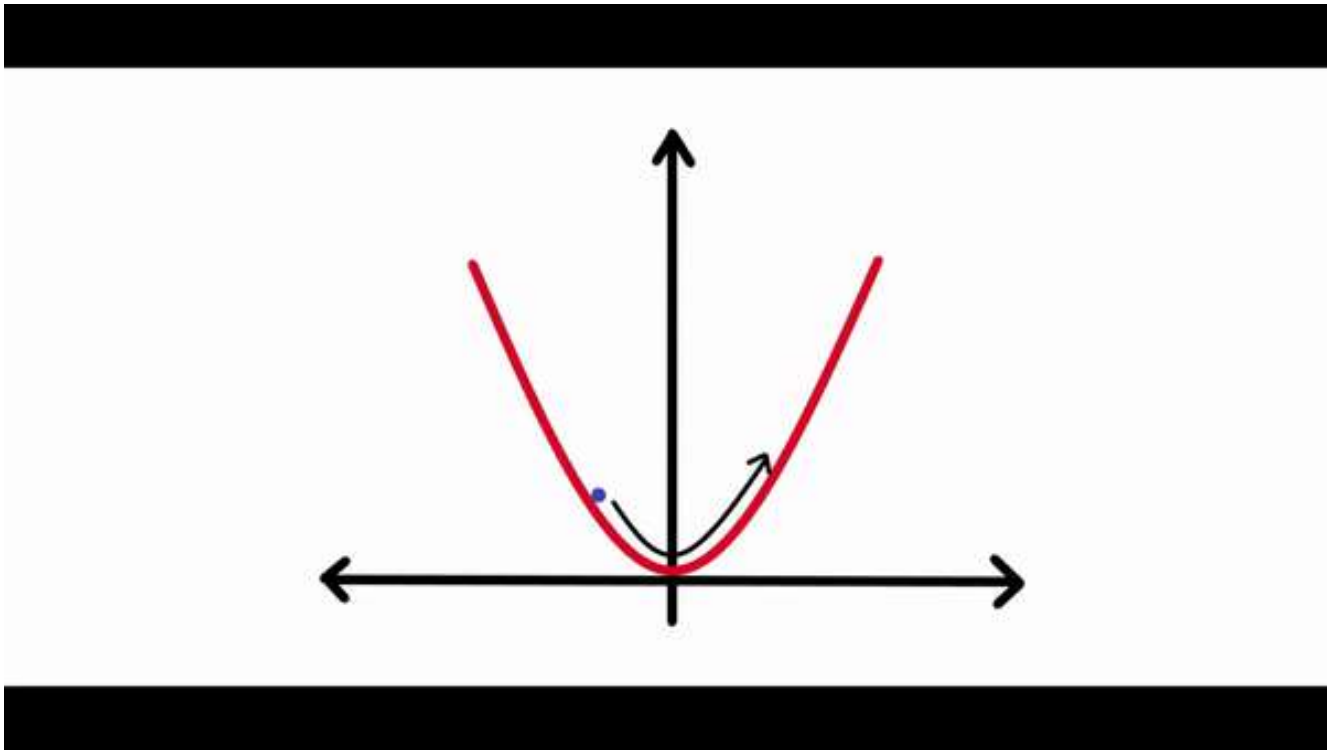
This is just because the loss keeps on changing at every point in these graphs, especially when the curve is a bit irregular. So what if we can make the neighborhood or the distance we consider to calculate the slope infinitesimally small. That would give us the most accurate gradient value possible.

That is exactly what we do by calculating the derivative of y with respect to x . This gives us the instantaneous rate of change in y with respect to x. The instantaneous rate of change gives us a more precise gradient than our previous rise by run approach just because this gradient is instantaneous.

The same approach is to be followed when calculating the rate of change of loss with respect to a weight or bias. The derivative of the loss function with respect to the weight gives us the instantaneous rate of change of loss with respect to the weight.

The Learning Rate:

After we calculate the gradient, we need to have something to scale down or scale up the gradient. Because sometimes when our neural network tries to march towards the lowest point in the loss curve, it may make large steps every time it adjusts it's weight and may never really converge to the global minimum. You can see what I mean in the graph below:



As you can see, the loss keeps on shooting in either direction and never really converges to the minimum.

At the same time, if the learning rate is too small, the loss may take years to converge to the minimum. So an optimal learning rate is crucial for any neural network to learn.

So we use the learning rate to scale down the size of the gradient every time a parameter gets updated. Let me reiterate the formula for updating the parameters we saw above.

```
new_weight = old_weight - learning_rate * gradient
```

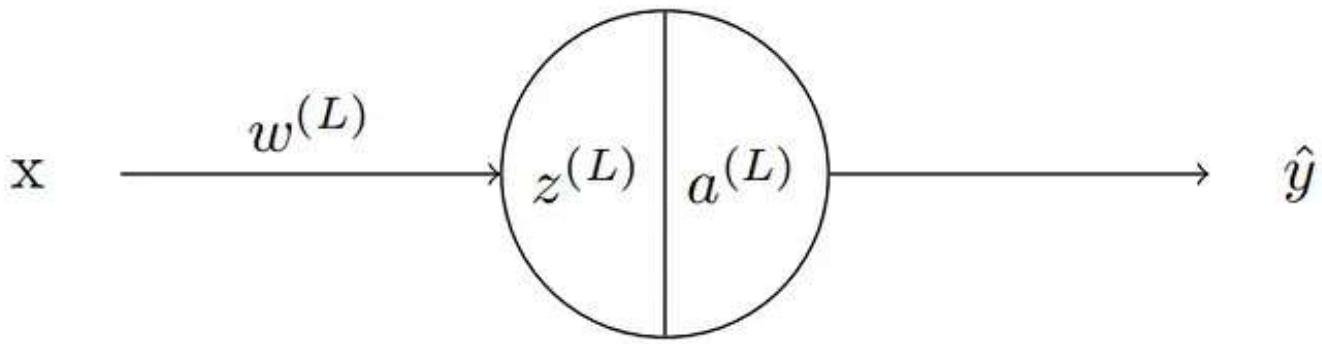
And so the learning rate determines the size of each step while converging to the minimum.

Calculating the Gradient:

The loss is attributed to the weights and biases of all the neurons in the network. Some weights may have influenced the output more than others and some may not have influenced the output at all.

Our goal now is to reduce the error in the output. But to achieve this we must calculate the gradient in each neuron. This gradient is then multiplied with the learning rate and subtracted from the current weight (or bias). This adjustment takes place in each and every neuron in the network.

Let us consider this neural network with only a single neuron.



where,
L- Layer number
w- weight
z- pre-activation function
a- activation function
y- output

The pre-activation z can be written as,

$$z^{(L)} = w^{(L)}x + b^{(L)}$$

Let us ignore the bias for now, for the sake of simplicity.

The value of z is then activated by an activation function. Let us consider the sigmoid activation function for this example. The sigmoid activation function is represented by the symbol σ.

$$\begin{aligned} z^{(L)} &= w^{(L)}x \\ a^{(L)} &= \sigma(z^{(L)}) \\ \hat{y} &= a^{(L)} \end{aligned}$$

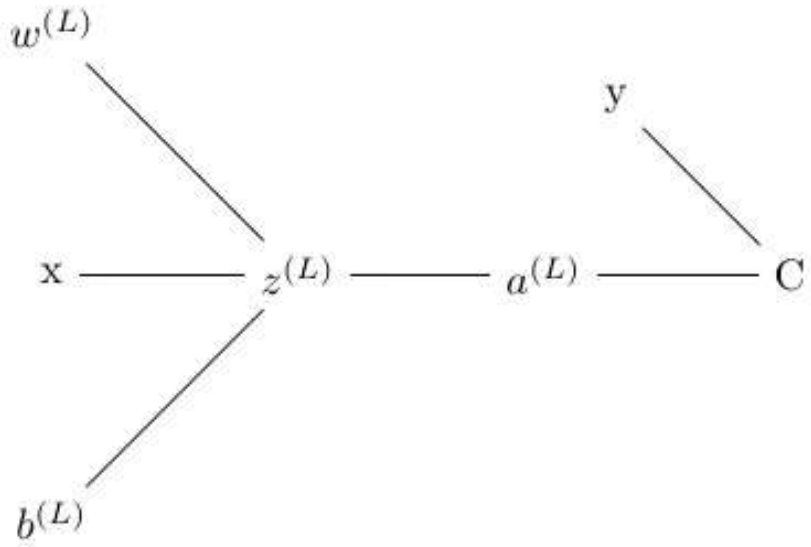
The output of this network is y-hat. Now the loss is calculated. This is done by using one of the various loss functions available. Let us represent the loss function with the letter C.

Now it's time to backpropagate stuff. The gradient of the loss function is calulated.

$$\text{Gradient} = \frac{\partial C}{\partial w}$$

This value basically tells us how any change in the weight affects the loss.

To calculate the gradient, we use the chain rule for finding derivatives. We use the chain rule because the error is not directly affected by the weight. The weight influences the pre-activation function which in turn affects the activation function which in turn affects the output and thus the loss. The tree below shows how each term depends on another term in the network above.



As you can see,

- the pre-activation function depends on the input, weight and bias.
- The activation function depends on the pre-activation function
- The loss depends on the activation function

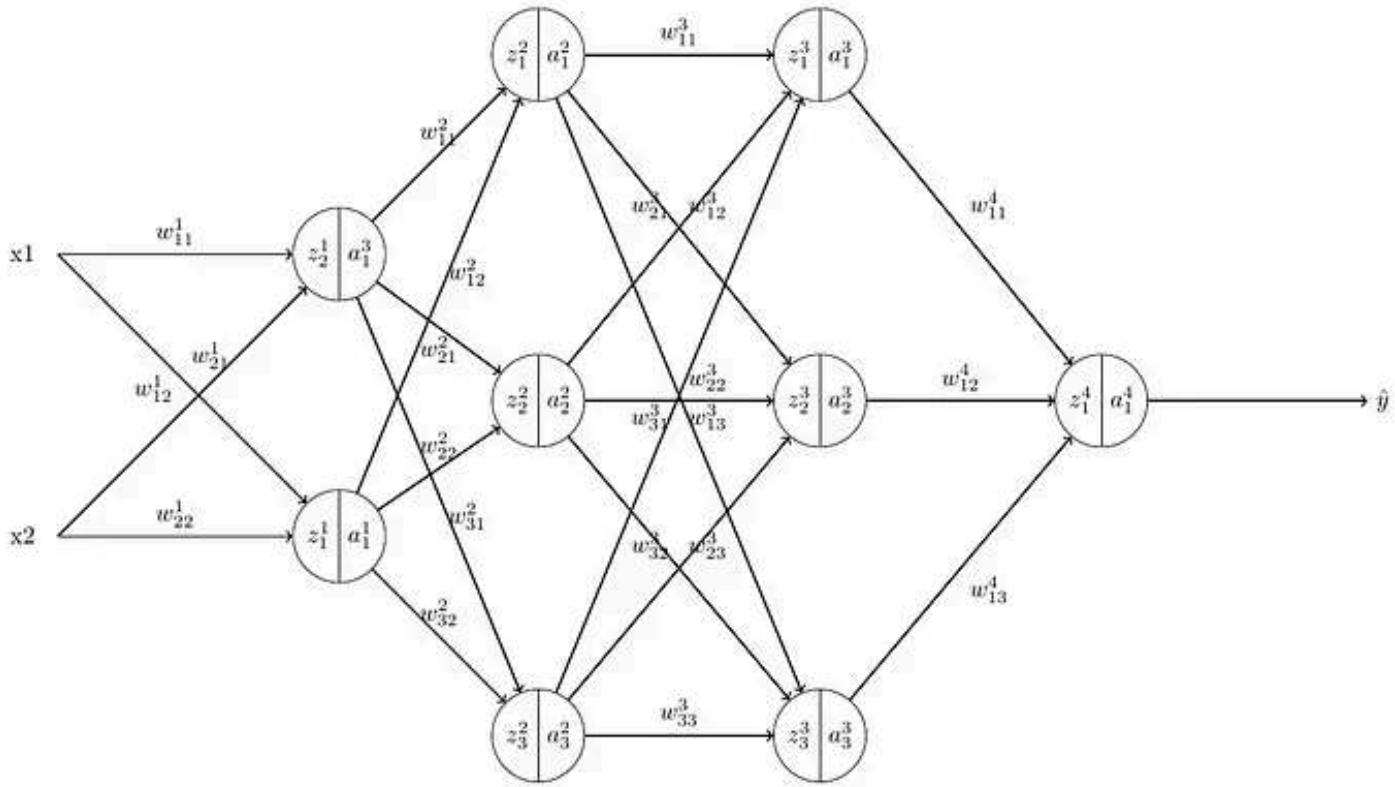
The y on the top right of the image is the ground truth with which the predicted output is compared and the loss is calculated.

So when we apply the chain rule, we get:

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial a^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

We have another phrase to refer to this gradient, the instantaneous rate of change of the loss with respect to the weight.

Now let us extrapolate this knowledge, gained from the gradient calculation of a single neuron network to a genuine neural network with four layers: an input layer, two hidden layers and an output layer.



The pre-activation function of each of these neurons is given by

$$z_j^{(L)} = w_{j0}^{(L)} a_0^{(L-1)} + w_{j1}^{(L)} a_0^{(L-1)} + w_{j2}^{(L)} a_0^{(L-1)} + \ldots + b_j^{(L)}$$

Where,
L- Layer number
j- index of the neuron for which the pre-activation function is being calculated
z- pre-activation function
w- weight of the neuron
a- activated output of the preceding neuron

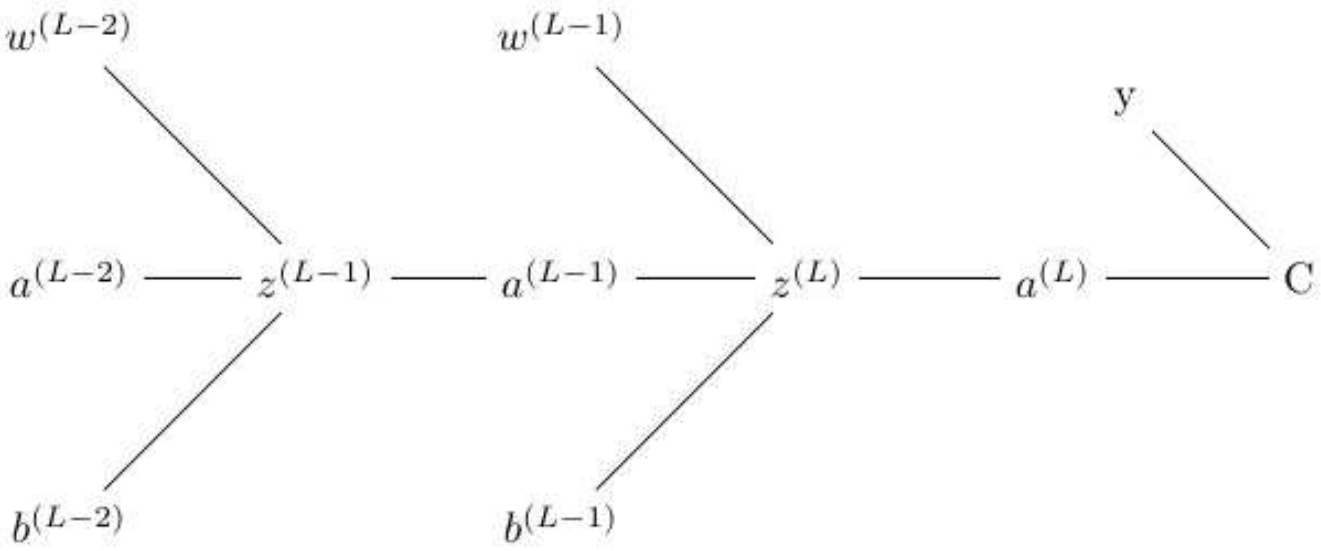
this is true for all the neurons except the input layer where we do not have an activation function. Therefore, in the input layer, z is just the sum of the inputs multiplied with their weights (not the previous neuron's activated output).

Here the gradient is given by,

$$\frac{\partial C}{\partial w_{jk}^{(L)}} = \frac{\partial C}{\partial a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \cdot \frac{\partial z_j^{(L)}}{\partial w_{jk}^{(L)}}$$

where w is the weight connecting the nodes k and j of layer L-1 and layer L respectively. k is the preceding node and j is the succeeding node. This might arise a new question. Then why is it wjk instead of wkj? Well it is just a naming convention to be followed while using matrices to multiply the weights with the inputs. But let us keep that aside for this article.

Have a look at the tree below to get an understanding of how these terms depend on each other.



Thus we can see that the output of the activation function of a node in the previous layer is given as the input of a node in the succeeding layer.

Now we can easily calculate the gradient in the output node if we know the values of the following terms:

- The derivative of the error with respect to the activation function
- The derivative of the activation function with respect to the pre-activation function
- The derivative of the pre-activation function with respect to the weight.

But when we calculate the gradient in the hidden layer, we will have to calculate the derivative of the loss function with respect to the activation function separately, before using it in the formula above.

$$\frac{\partial C}{\partial a_k^{(L-1)}} = \sum_{j=0}^{n_{(L-1)}} \frac{\partial C}{\partial a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \cdot \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}}$$

This equation is pretty much same as the first one (the derivation of the loss function with respect to the weight). But we have a summation here. This is because unlike a weight, the activation function of one neuron can affect the outcomes of all the neurons, it is connected to, in the succeeding layer.

We do not need to write a separate equation with the chain rule applied for calculating the derivation of the loss function with respect to the activation function in the **output layer**. That's because the activation function in the output layer directly affects the error. But that's not the case with the activation functions of the hidden and input layers. They indirectly affect the final output by taking different paths through the network.

Thus after calculating the gradient in all the nodes in the network, it is multiplied with the learning rate and subtracted from the corresponding weights.

This is how the error is backpropagated and the weights are adjusted. After several iterations of this process, the loss gets reduced to the global minimum and eventually the training comes to an end.

Wait a minute.. what about the bias?

Well the bias too undergoes everything the same way as the weight did!

Like the weight the bias too affects the output of the network. And so in each training iteration, the gradient is calculated for the loss with respect to the bias simultaneously when the gradient is being calculated for the loss with respect to the weight.

$$\frac{\partial C}{\partial b_j^{(L)}} = \frac{\partial C}{\partial a_j^{(L)}} \cdot \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \cdot \frac{\partial z_j^{(L)}}{\partial b_j^{(L)}}$$

Here too for the hidden layers, the derivation of the loss function with respect to the activation function of the previous layer is to be calculated separately using the chain rule.

Thus the gradient is backpropagated and the bias in each node is adjusted.

That's it! That is all that happens under the hood during each training loop when the loss is being backpropagated and minimized. I hope this has cleared up the obscurity in the math so that the next time you hit that reverse gear in your neural network and something doesn't feel right, you'll know just where to look into.

Thank you!

Backpropagation | Deep Learning | Artificial Neural Network | Neural Networks | Mathematics